



**Universidade de Brasília**

Departamento de Ciência da Computação

# RISC-V Pipeline: Conceitos

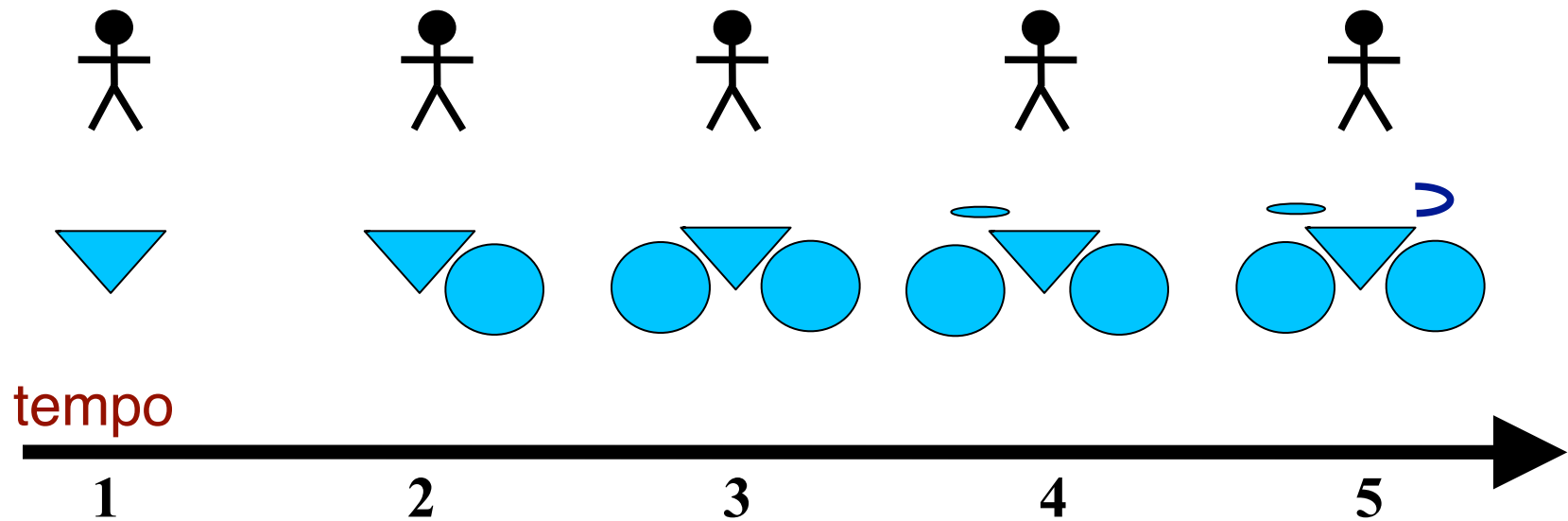


# Objetivo

- Apresentar técnicas para aumento de desempenho de uma única tarefa em único processador usando ILP (*Instruction Level Parallelism*):
  - **Pipelining** : Técnica de projeto onde o *hardware* processa mais de uma instrução de cada vez, sem esperar que uma instrução termine antes de começar a próxima.
  - **Superpipeline** : Técnicas para aumento da eficiência do pipeline.
  - **Superescalar**: Uso eficiente de múltiplas unidades funcionais.
- Outros níveis de paralelismo: Aumenta a eficiência em máquinas multi task.
  - Multithread : Várias tarefas independentes simultaneamente, aproveitando-se das “falhas” do pipeline e/ou de estrutura superescalar.
  - Multicore: Vários núcleos em 1 chip, compartilhando memórias caches.
  - Multiprocessor: Sistema multiprocessado/distribuído. Diferentes computadores.



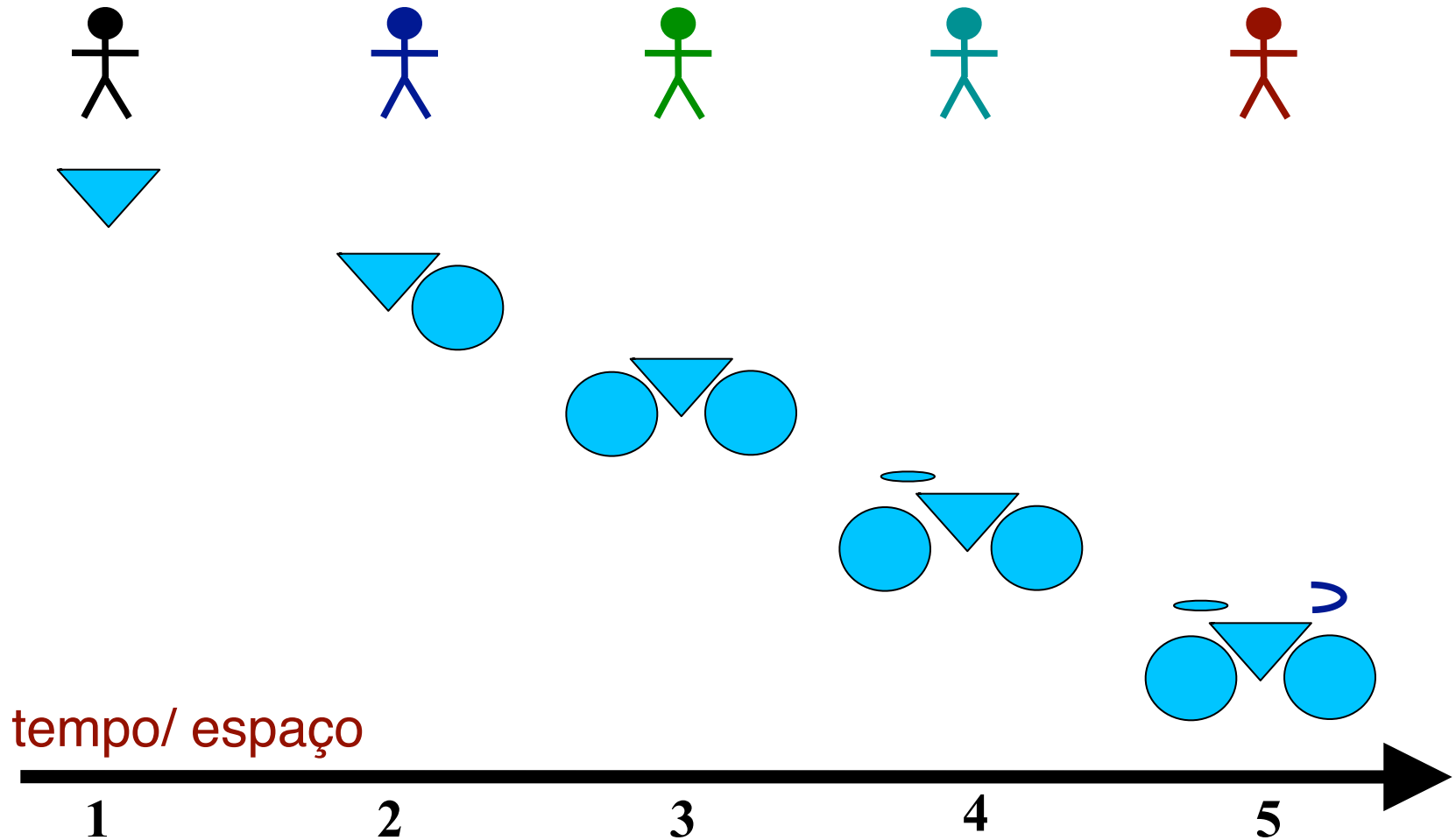
# Pipelining: Conceito Básico



- uma pessoa leva 5 unidades de tempo para montar uma bicicleta



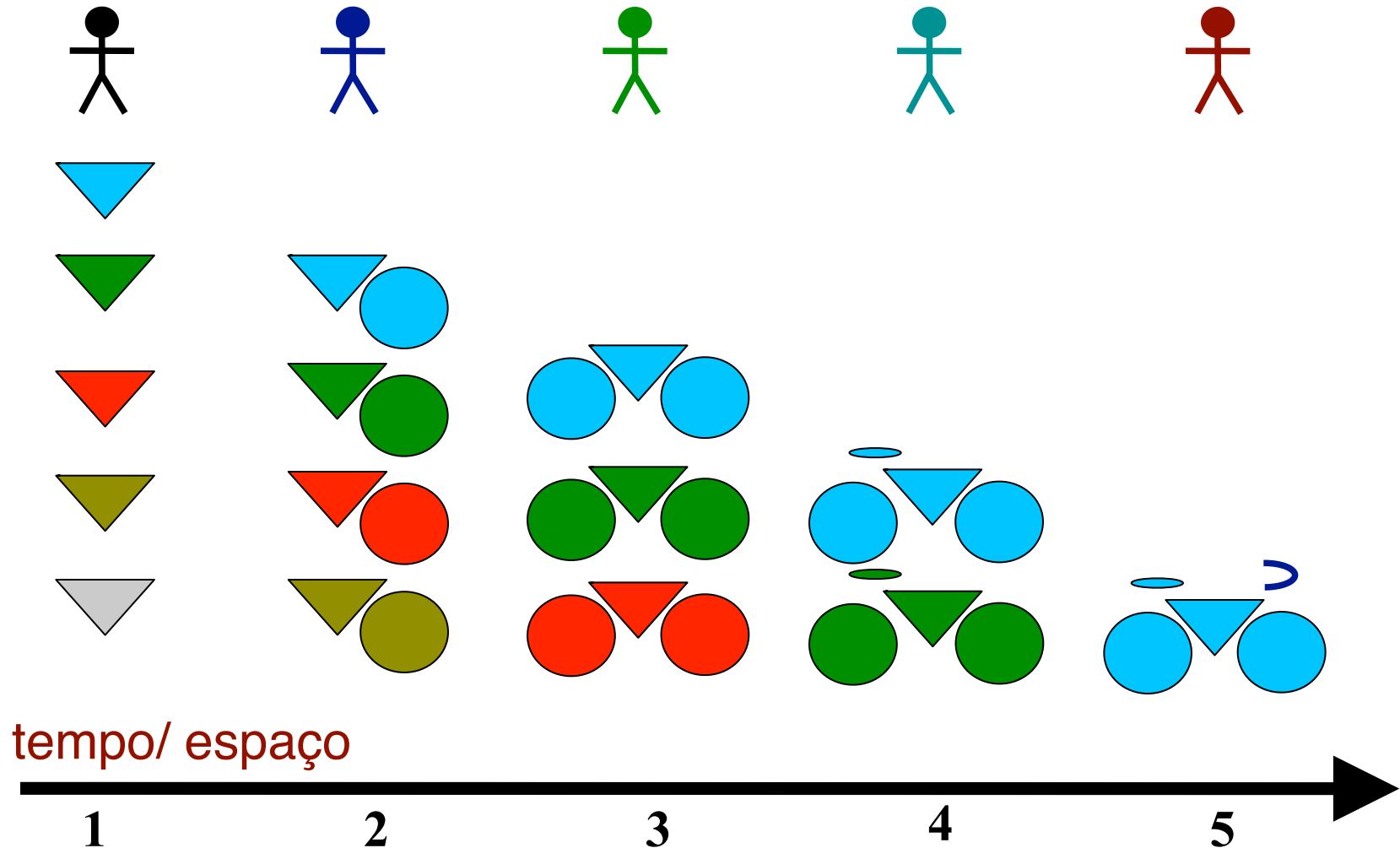
# Linha de Montagem (pipeline)



- uma pessoa diferente para cada tarefa



S





# Análise

- Latência:
  - tempo que uma bicicleta leva para ser montada
  - 5 ciclos
- Vazão:
  - número de bicicletas por unidade de tempo
  - 1 bike/ciclo
- Qual a aceleração ideal?
  - Para estágios balanceados, condições ideais e grande número de instruções:

$$\textit{Tempo entre instruções}_{com\ pipeline} = \frac{\textit{Tempo entre instruções}_{sem\ pipeline}}{\textit{Número de estágios do pipeline}}$$



# Análise

- O *pipeline* só começa a produzir nessa taxa uma vez que linha esteja cheia
- Primeira bicicleta produzida em 5 UT
- Após a primeira, o pipe produz uma bicicleta por UT
- Balanceamento
  - Suponha que a operação Montagem da Roda Traseira requeira duas unidades de tempo
    - Uma unidade para montar a correia
    - Uma unidade para montar a roda
  - O que ocorre com a latência do pipeline ?
  - O que ocorre com a vazão ?
  - Como otimizar o pipeline neste caso ?



# Balanceamento

- O pipeline não funciona se as tarefas não requerem o mesmo tempo
- Se uma tarefa leva 2 UT, todas tem que trabalhar neste período
  - Latência do pipeline seria 10 UT
  - Vazão seria uma bicicleta / 2 UT
- Dividindo a tarefa em duas subtarefas de 1 UT cada, teríamos:
  - Latência de 6 UT
  - Vazão de 1 Bike / UT





# Pipeline do RISC-V

- Cinco estágios, uma fase por estágio
  1. IF (*instruction fetch*): busca da instrução
  2. ID (*instruction decode*) : identificação da instrução e leitura dos registradores
  3. EX (*execute*) : execução da instrução ou cálculo dos endereços
  4. MEM (*memory*) : operação de acesso à memória
  5. WB (*write back*) : escreve resultado em registrador



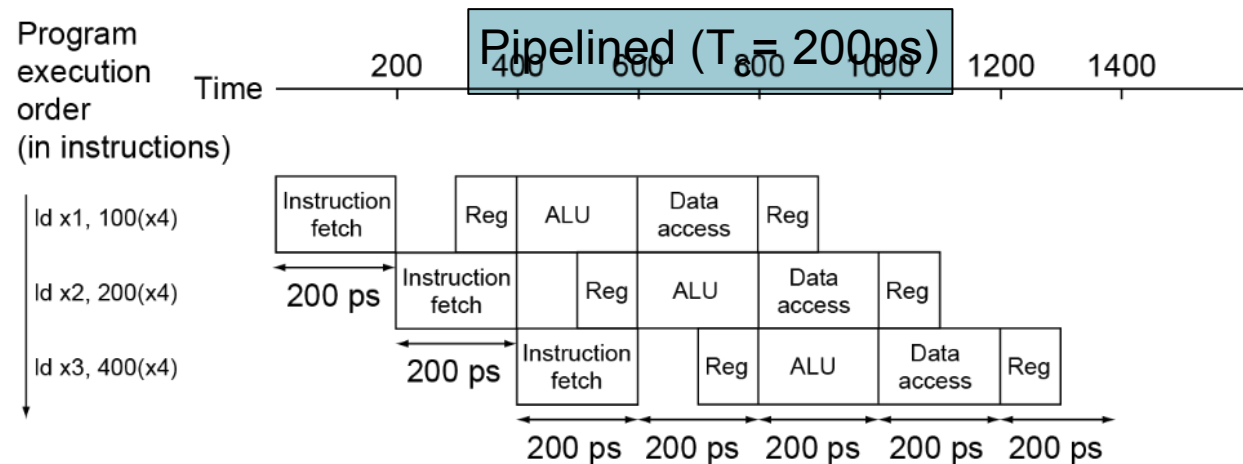
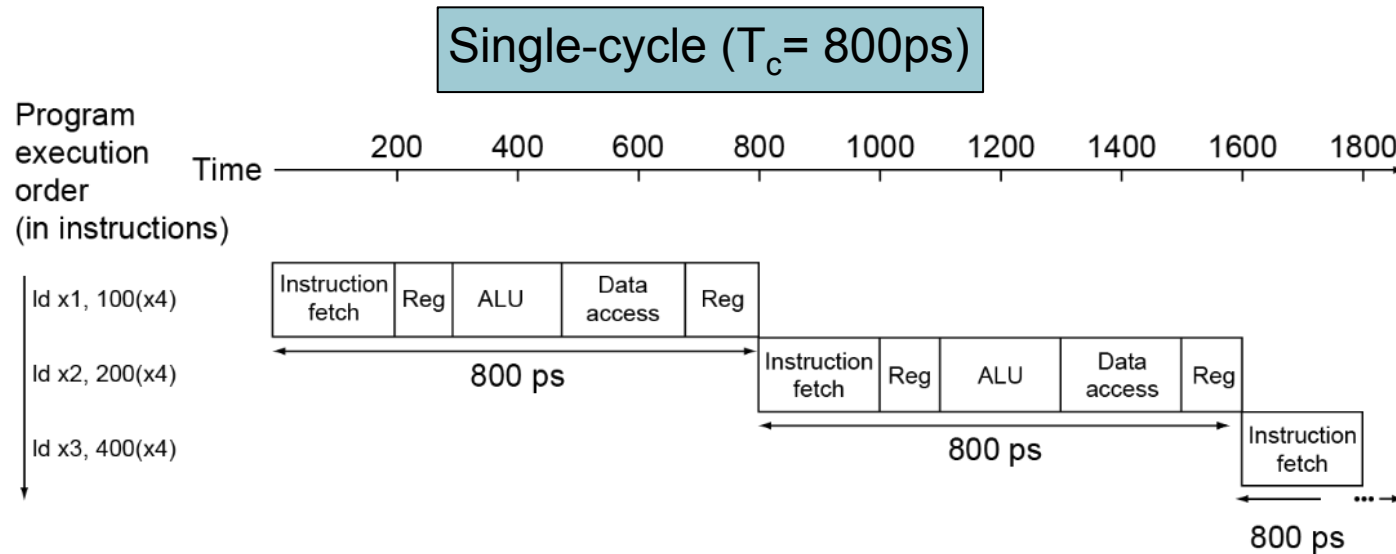
# Desempenho do Pipeline

- Assumir que o atraso dos estágios é
  - 100 ps para acesso aos registradores
  - 200 ps para os outros estágios
- Comparar o desempenho entre as unidades uniciclo e pipeline

| Instr    | Instr fetch | Register read | ALU op | Memory access | Register write | Total time |
|----------|-------------|---------------|--------|---------------|----------------|------------|
| ld       | 200ps       | 100 ps        | 200ps  | 200ps         | 100 ps         | 800ps      |
| sd       | 200ps       | 100 ps        | 200ps  | 200ps         |                | 700ps      |
| R-format | 200ps       | 100 ps        | 200ps  |               | 100 ps         | 600ps      |
| beq      | 200ps       | 100 ps        | 200ps  |               |                | 500ps      |



# Uniciclo vs Pipeline





## ■ Exemplo slide anterior:

### □ Ideal:

$$Tempo\ entre\ instruções_{com\ pipeline} = \frac{Tempo\ entre\ instruções_{sem\ pipeline}}{Número\ de\ estágios\ do\ pipeline} = \frac{800}{5} = 160$$

□ Porém → tempo entre instruções do pipeline → 200

$$200 = \frac{Tempo\ entre\ instruções_{sem\ pipeline}}{Número\ de\ estágios\ do\ pipeline} \quad Num.\ estágios\ pipeline = \frac{800}{200} = 4$$

### □ 3 instruções:

- Sem pipeline : 2400
- Com pipeline : 1400
- **Aceleração → 2400/1400 = 1,7**

### □ 1000 instruções:

- Sem pipeline : 1000 x 800 = 800.000
- Com pipeline: 1x1000 + 999x200 = 1000 + 199800 = 200.800
- **Aceleração: 800.000/200.600 = 3,98**



# Pipeline

## O que facilita o pipeline:

- Todas instruções têm o mesmo comprimento.

Obs.: IA-32 instruções de 1 a 17 bytes

- Poucos formatos de instruções.

Obs: Tipo-R, I , J operandos “quase” nas mesmas posições

- Operandos em memória só aparecem em *loads* e *stores*.

Obs.: Pode-se usar a ULA para cálculo de endereço.

O mesmo não vale para busca de operandos da ULA da memória (IA-32)

## O que complica (hazards):

- Conflitos Estruturais
- Conflitos de Controle
- Conflitos de Dados



# Conflitos Estruturais

- Hardware pode não admitir a combinação de instruções em um mesmo ciclo de clock.
  - uma pessoa apenas para montar as bicicletas
  - Considerando apenas 1 memória para dados e instruções, como ficaria uma quarta instrução no exemplo do *slide* 9?



# Conflitos de Dados

- Pipeline precisa ser interrompido por que uma etapa precisa esperar até que outra seja concluída (dependência de dados).

□ Exemplo:

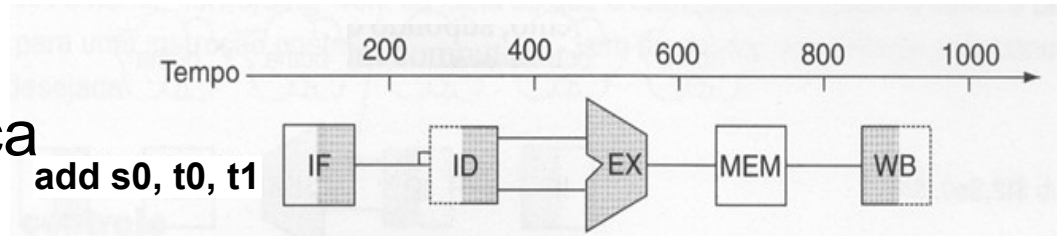
- add s0,t0,t1
- sub t2,s0,t3

Soluções:

- Inserção de Bolhas
- Reordenamento de código (compilador e processador)
- Forwarding ou bypassing

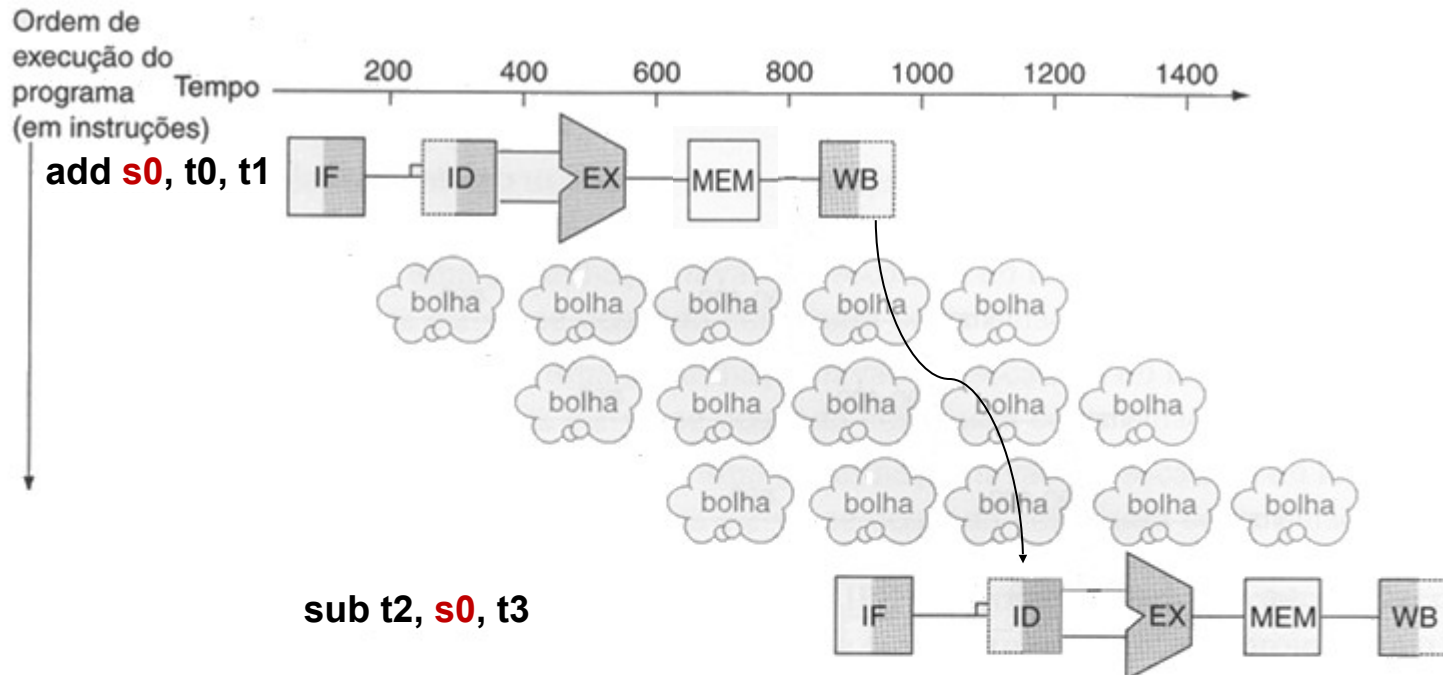
# Inserção de Bolhas

- Representação gráfica



- Bolhas

- Sabendo que o registrador `s0` é escrito no 5º estágio







# Inserção de Bolhas

- Representação tradicional

| Ciclo          | 01 | 02 | 03 | 04                | 05                | 06                | 07 | 08 | 09 | 10 | 11 | 12 | 13 |
|----------------|----|----|----|-------------------|-------------------|-------------------|----|----|----|----|----|----|----|
| add s0, t0, t1 | IF | ID | Ex | Me                | WB                |                   |    |    |    |    |    |    |    |
| sub t2, s0, t3 |    | IF | ID | * <sub>(ID)</sub> | * <sub>(ID)</sub> | * <sub>(ID)</sub> | Ex | Me | WB |    |    |    |    |
|                |    |    |    |                   |                   |                   |    |    |    |    |    |    |    |
|                |    |    |    |                   |                   |                   |    |    |    |    |    |    |    |

3 ciclos de stall



# Execução Fora de Ordem

- Aproveitar os espaços das bolhas para tarefas úteis que sejam independentes.
- Alterar o algoritmo sem alterar o resultado!

□ Ex.:

```
ori t7,t7,100  
add s1,zero,zero  
addi t5,t4,256  
add s0, t0, t1  
sub t2, s0, t3
```

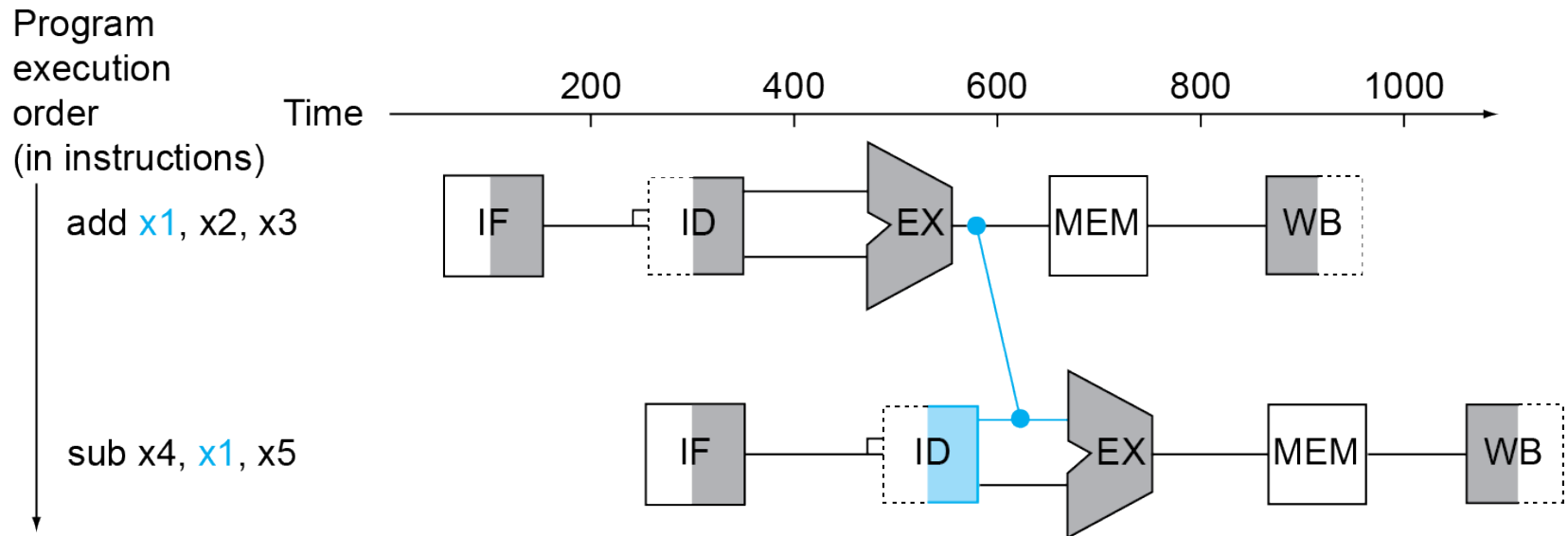


```
add s0, t0, t1  
ori t7, t7, 100  
add s1, zero, zero  
addi t5, t4, 256  
sub t2, s0, t3
```



# Adiantamento de Dados

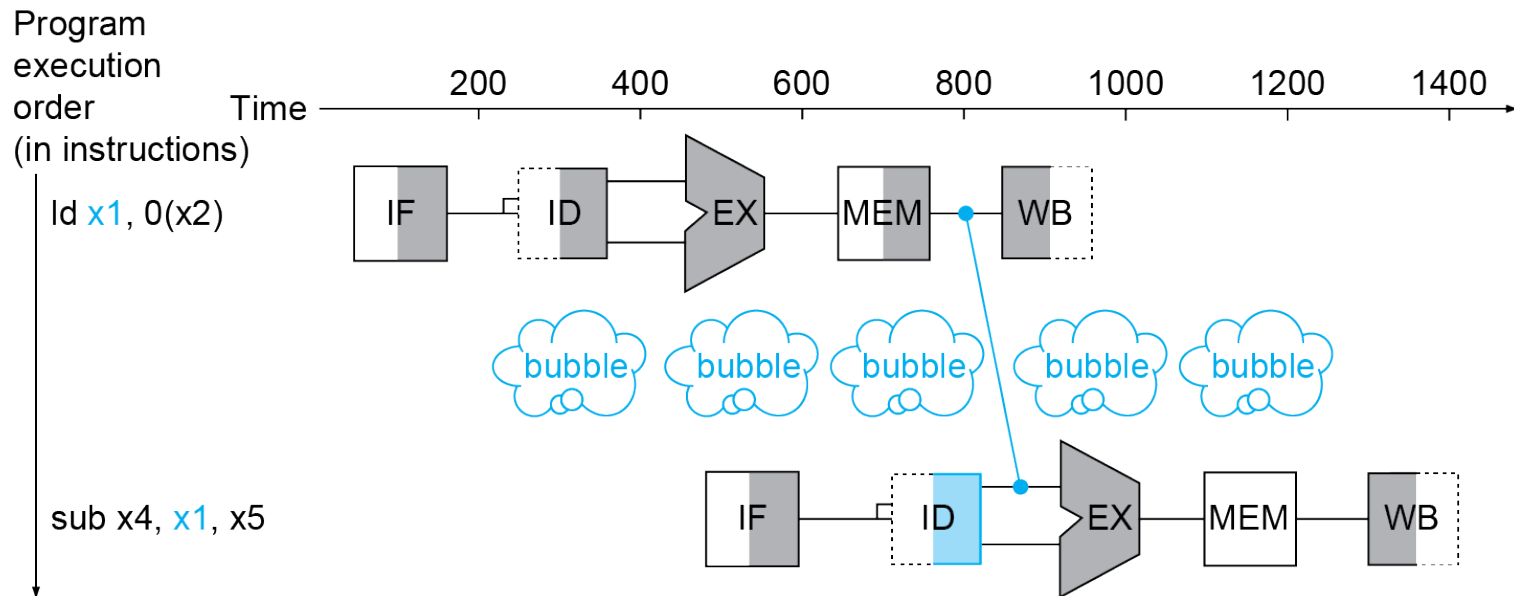
- Observação: não é necessário esperar que a instrução termine antes de tentar resolver o hazard de dados.





# Adiantamento de Dados

- E se fosse um load?
  - Não é possível adiantar um dado que não está disponível
  - Não se pode voltar no tempo!





■ Exemplo:

A=B+E;

C=B+F;

lw t1, 0(t0)

lw t2, 4(t0)

add t3, t1, t2

sw t3, 12(t0)

lw t4, 8(t0)

add t5, t1, t4

sw t5, 16(t0)

Identifique os hazards.

Como solucionar utilizando apenas bolhas?

Quantos ciclos são necessários?

Como solucionar de maneira mais eficiente?

Quantos ciclos são necessários?



## ■ Sequência original

|                | 1  | 2  | 3  | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11 |
|----------------|----|----|----|-----|-----|-----|-----|-----|-----|-----|----|
| lw t1, 0(t0)   | IF | ID | EX | MEM | WB  |     |     |     |     |     |    |
| lw t2, 4(t0)   |    | IF | ID | EX  | MEM | WB  |     |     |     |     |    |
| add t3, t1, t2 |    |    | IF | ID  | EX  | MEM | WB  |     |     |     |    |
| sw t3, 12(t0)  |    |    |    | IF  | ID  | EX  | MEM | WB  |     |     |    |
| lw t4, 8(t0)   |    |    |    |     | IF  | ID  | EX  | MEM | WB  |     |    |
| add t5, t1, t4 |    |    |    |     |     | IF  | ID  | EX  | MEM | WB  |    |
| sw t5, 16(t0)  |    |    |    |     |     |     | IF  | ID  | EX  | MEM | WB |



## ■ Solução com *nops*

|                | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 |
|----------------|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|----|
| lw t1, 0(t0)   | IF | ID | EX | ME | WB |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| lw t2, 4(t0)   |    | IF | ID | EX | ME | WB |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| add t3, t1, t2 |    |    |    |    |    | IF | ID | EX | ME | WB |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| sw t3, 12(t0)  |    |    |    |    |    |    |    |    |    | IF | ID | EX | ME | WB |    |    |    |    |    |    |    |    |    |
| lw t4, 8(t0)   |    |    |    |    |    |    |    |    |    |    | IF | ID | EX | ME | WB |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| add t5, t1, t4 |    |    |    |    |    |    |    |    |    |    |    |    |    |    | IF | ID | EX | ME | WB |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| NOP            |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |
| sw t5, 16(t0)  |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    | IF | ID | EX | ME | WB |
|                |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |    |



## ■ Solução com adiantamento de dados (*forwarding*)

|                                       | 1  | 2  | 3  | 4   | 5   | 6  | 7   | 8   | 9   | 10  | 11  | 12 | 13 |
|---------------------------------------|----|----|----|-----|-----|----|-----|-----|-----|-----|-----|----|----|
| lw <b>t1</b> , 0(t0)                  | IF | ID | EX | MEM | WB  |    |     |     |     |     |     |    |    |
| lw <b>t2</b> , 4(t0)                  |    | IF | ID | EX  | MEM | WB |     |     |     |     |     |    |    |
| <b>BOLHA</b>                          |    |    |    |     |     |    |     |     |     |     |     |    |    |
| add <b>t3</b> , <b>t1</b> , <b>t2</b> |    |    |    | IF  | ID  | EX | MEM | WB  |     |     |     |    |    |
| sw <b>t3</b> , 12(t0)                 |    |    |    |     | IF  | ID | EX  | MEM | WB  |     |     |    |    |
| lw <b>t4</b> , 8(t0)                  |    |    |    |     |     | IF | ID  | EX  | MEM | WB  |     |    |    |
| <b>BOLHA</b>                          |    |    |    |     |     |    |     |     |     |     |     |    |    |
| add <b>t5</b> , t1, <b>t4</b>         |    |    |    |     |     |    | IF  | ID  | EX  | MEM | WB  |    |    |
| sw <b>t5</b> , 16(t0)                 |    |    |    |     |     |    |     | IF  | ID  | EX  | MEM | WB |    |

**t1**: escrita e leitura no banco de registradores no mesmo ciclo

**t2**: estágio *writeback* para estágio de execução do **add**

**t3**: estágio *writeback* do **add** para o estágio de escrita na memória do **sw**

**t4**: estágio *writeback* do **lw** para o estágio de execução do **add**

**t5**: estágio *writeback* do **add** para o estágio de escrita na memória do **sw**





## ■ Solução com adiantamento de dados (*forwarding*) e reordenamento de instruções

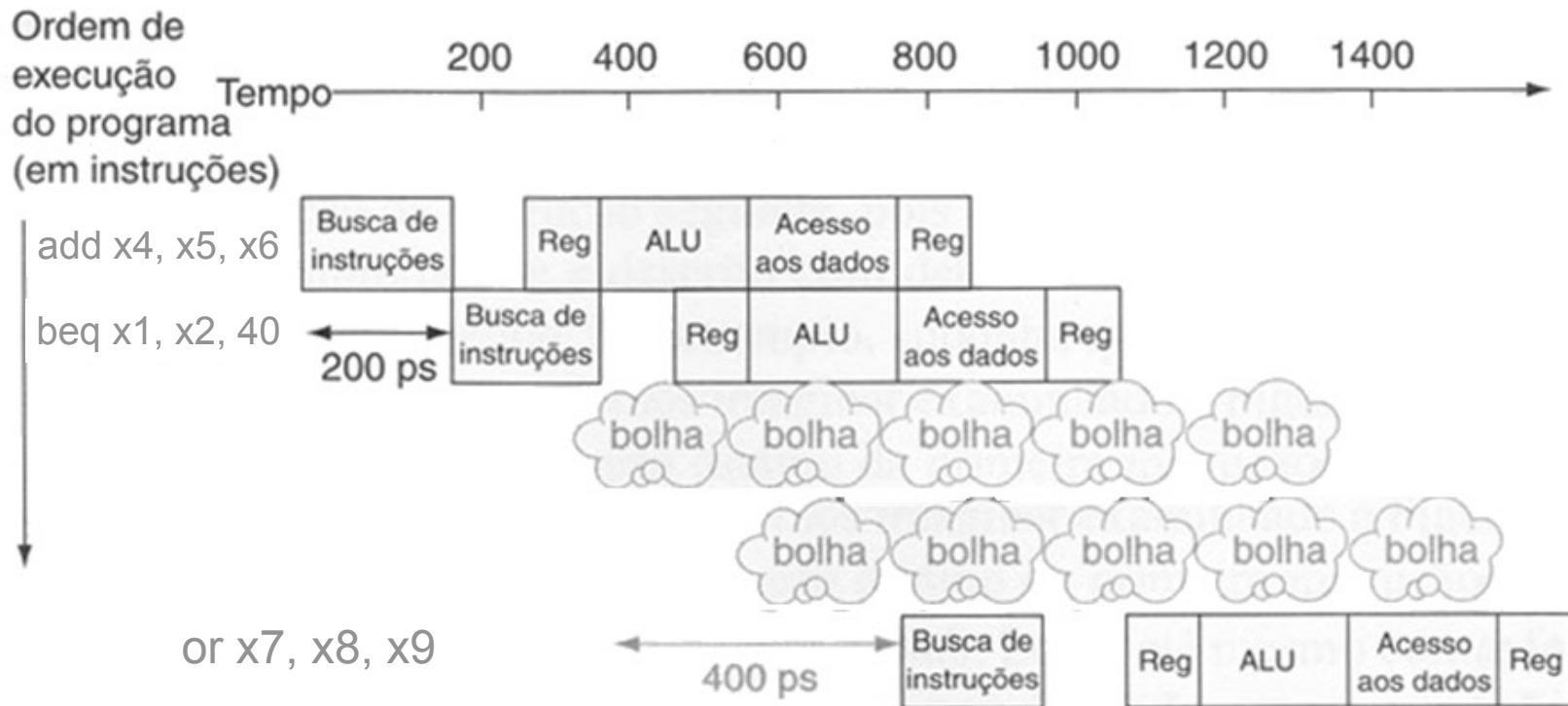
|                | 1  | 2  | 3  | 4   | 5   | 6   | 7   | 8   | 9   | 10  | 11 |
|----------------|----|----|----|-----|-----|-----|-----|-----|-----|-----|----|
| lw t1, 0(t0)   | IF | ID | EX | MEM | WB  |     |     |     |     |     |    |
| lw t2, 4(t0)   |    | IF | ID | EX  | MEM | WB  |     |     |     |     |    |
| lw t4, 8(t0)   |    |    | IF | ID  | EX  | MEM | WB  |     |     |     |    |
| add t3, t1, t2 |    |    |    | IF  | ID  | EX  | MEM | WB  |     |     |    |
| sw t3, 12(t0)  |    |    |    |     | IF  | ID  | EX  | MEM | WB  |     |    |
| add t5, t1, t4 |    |    |    |     |     | IF  | ID  | EX  | MEM | WB  |    |
| sw t5, 16(t0)  |    |    |    |     |     |     | IF  | ID  | EX  | MEM | WB |

lw t1, 0(t0)  
 lw t2, 4(t0)  
 ↗ add t3, t1, t2  
 ↘ sw t3, 12(t0)  
 lw t4, 8(t0)  
 add t5, t1, t4  
 sw t5, 16(t0)



# Conflito de Controle

- Necessidade de tomar uma decisão com base nos resultados de uma instrução enquanto outras estão sendo executadas.

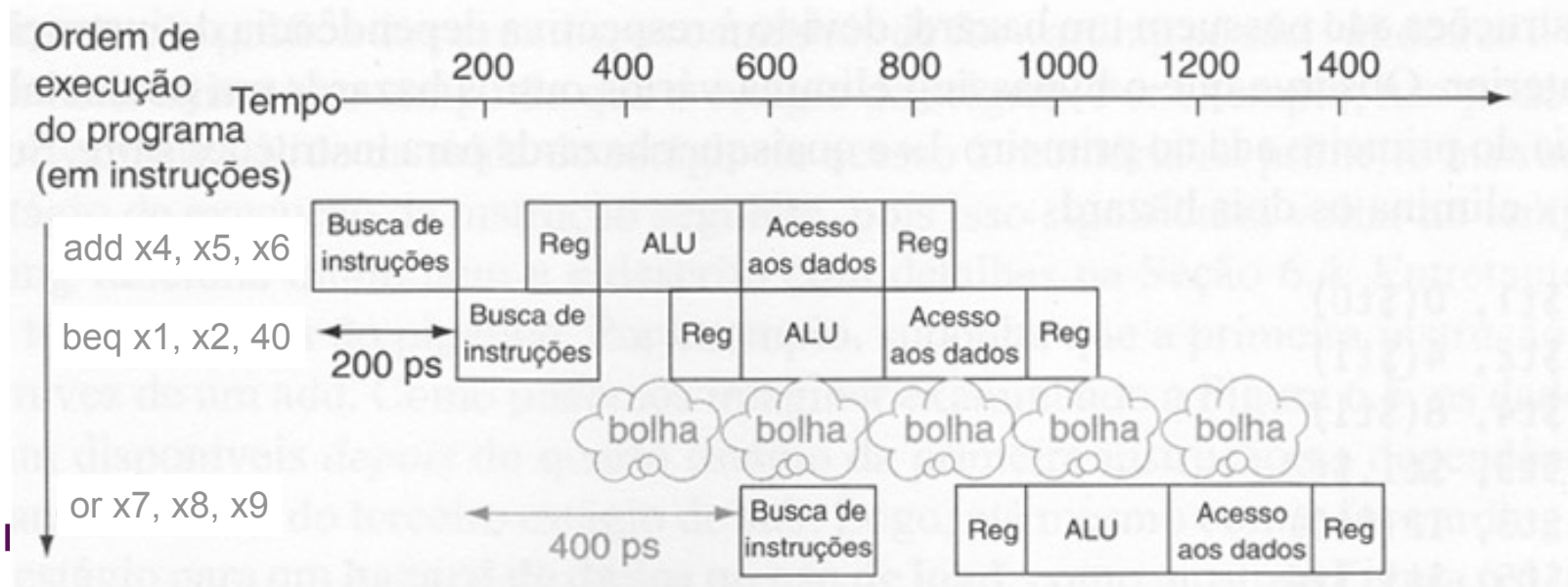


BEQ é avaliado pela ULA no 3º estágio => Necessita 2 bolhas!!!



# Conflito de Controle

- Alterando o caminho de dados para que o BEQ seja avaliado no 2º Estágio => Economiza 1 Bolha!!

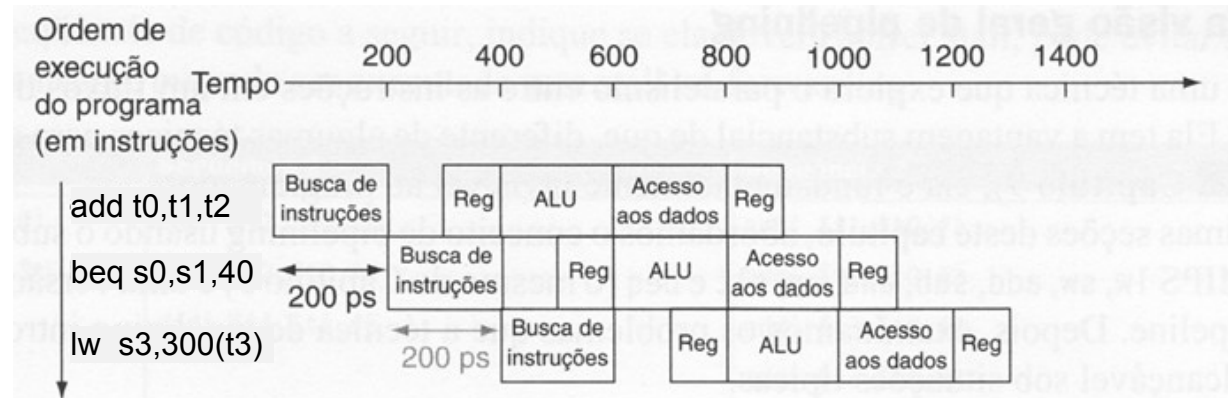




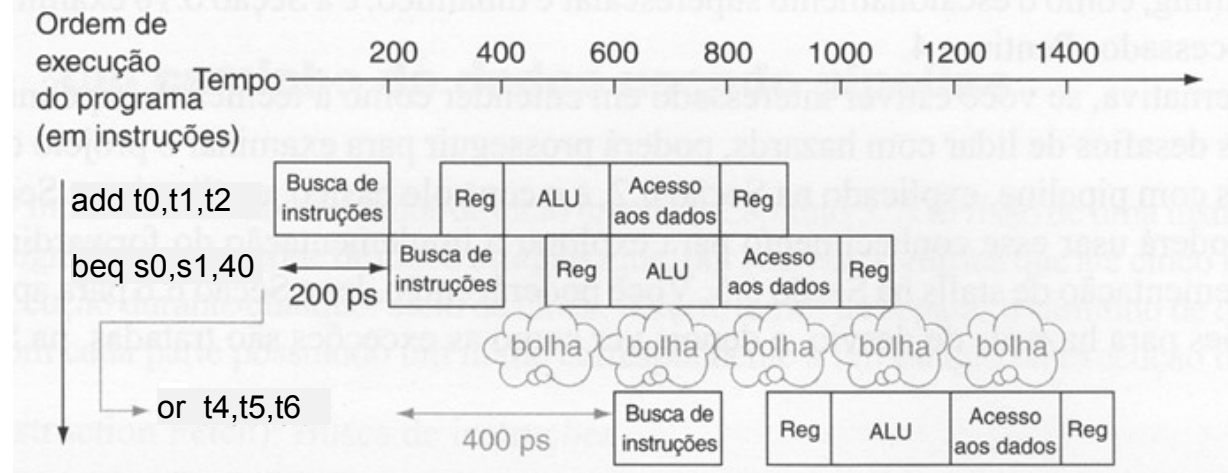
# Conflito de Controle: Previsão de Desvios

## ■ Prevendo estaticamente o desvio como não tomado

Caso de:  
Desvio não tomado  
Acerto!



Caso de:  
Desvio tomado  
Erro! Necessita dar um *flush* na instrução seguinte





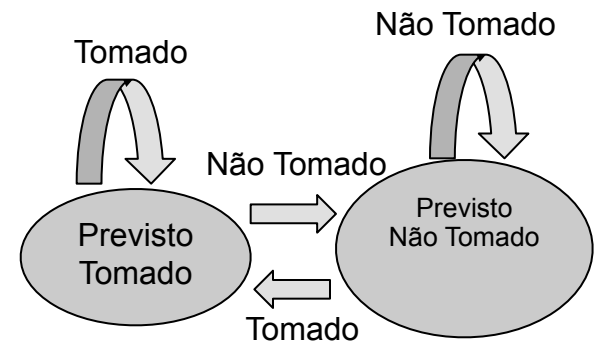
# Hazard de Controle: Previsão de Desvios

## ■ Prevendo Dinamicamente o desvio

•Ex.: Endereço é anterior? Prevê tomar o desvio

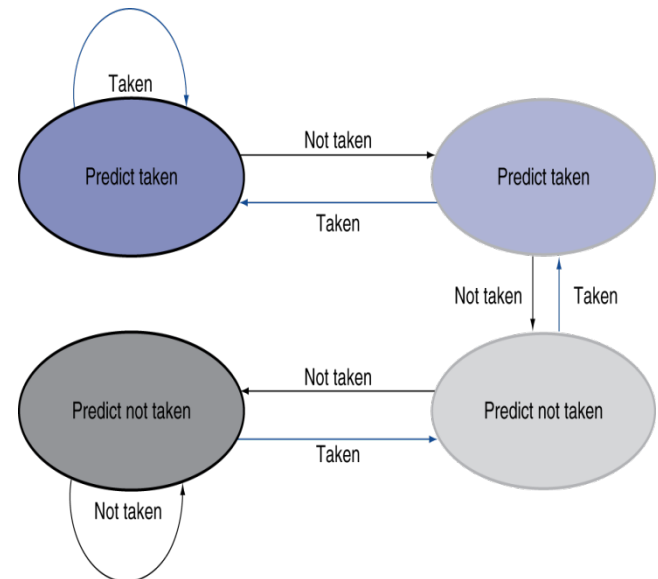
### ■ 1 Bit

□ Considera que a previsão é a mesma da última escolha



### ■ 2 Bits

□ Considera uma variação conforme:



### ■ Tabela

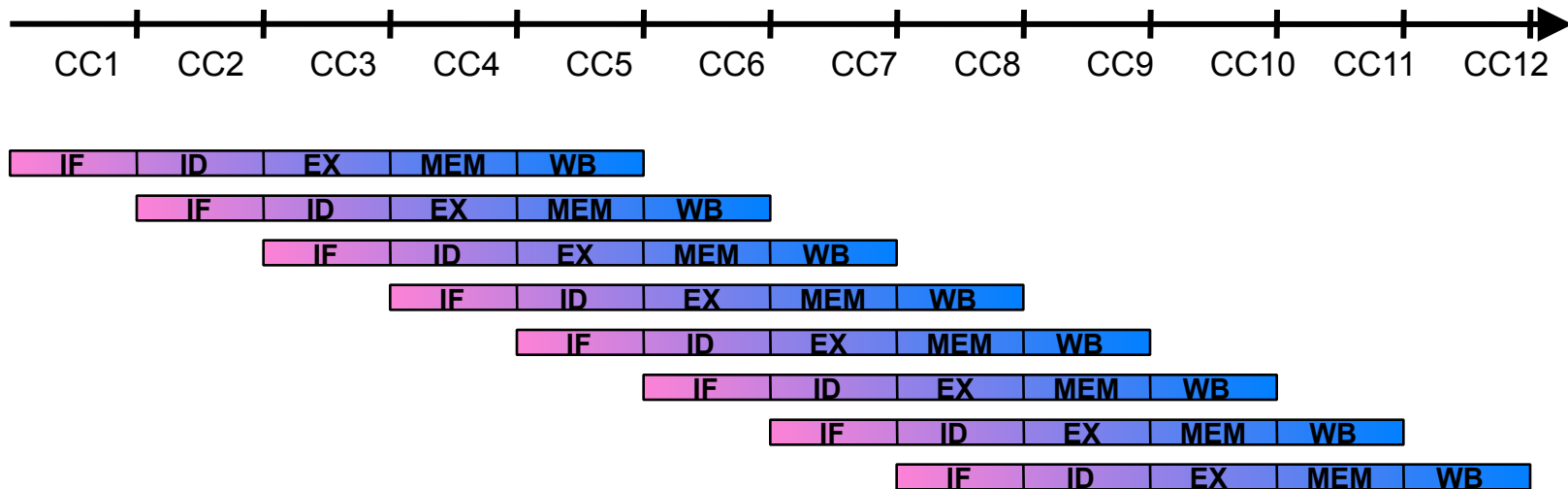
□ Considera uma tabela com contador do histórico

Mostrar Tool / BHT Simulator do RARS



# Acelerando máquinas com *pipeline*

- A velocidade máxima de uma máquina com *pipeline* é de uma instrução por ciclo de *clock*. Correto?



Técnicas para iniciar múltiplas instruções em 1 ciclo de clock:

- Estruturas *superpipeline*
- Estruturas *superescalares*.

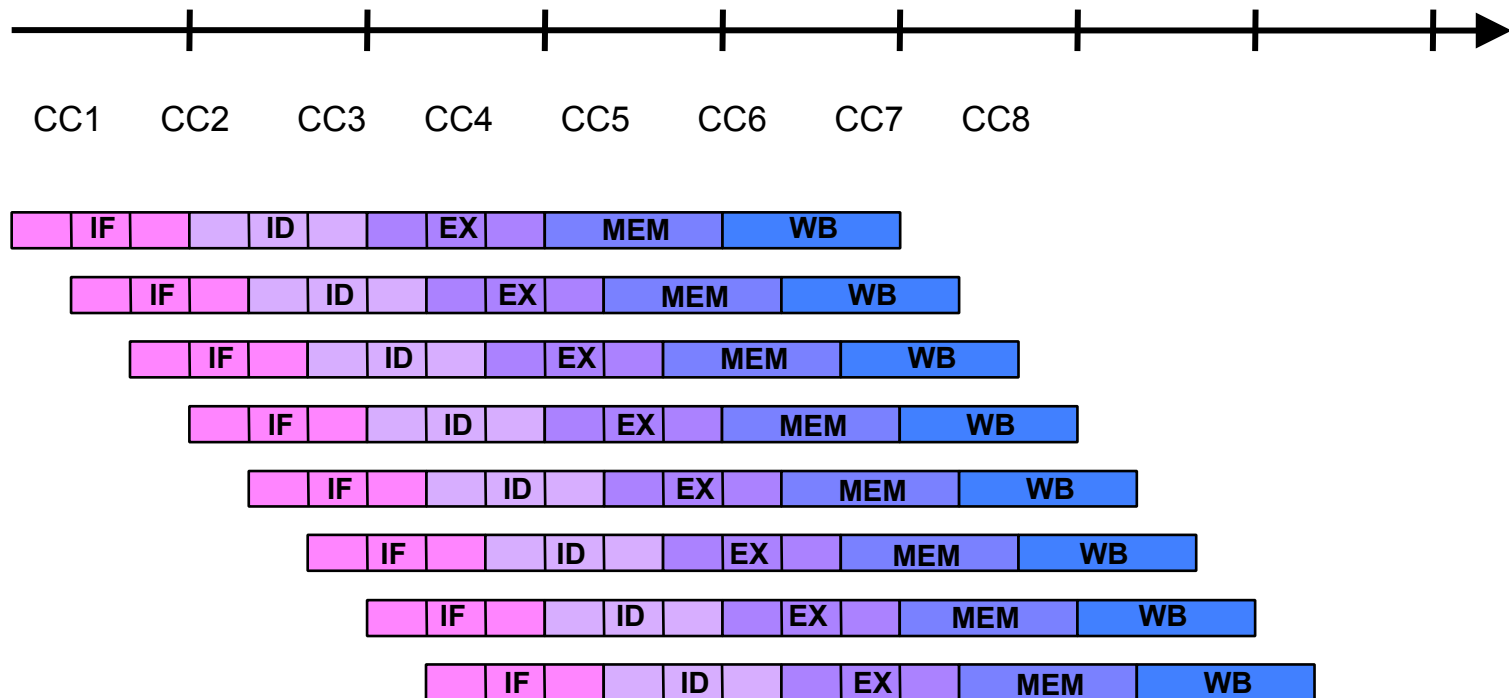


# Acelerando máquinas: *Superpipeline*

- A estrutura *Superpipeline* subdivide cada estágio do *pipeline* em subestágios, e multiplica internamente a frequência de *clock*.
- Cada (sub)estágio continua executando uma instrução por *clock*. Mas como o *clock* interno é multiplicado, o *pipeline* pode aceitar duas ou mais instruções para cada *clock* externo.



# Superpipeline





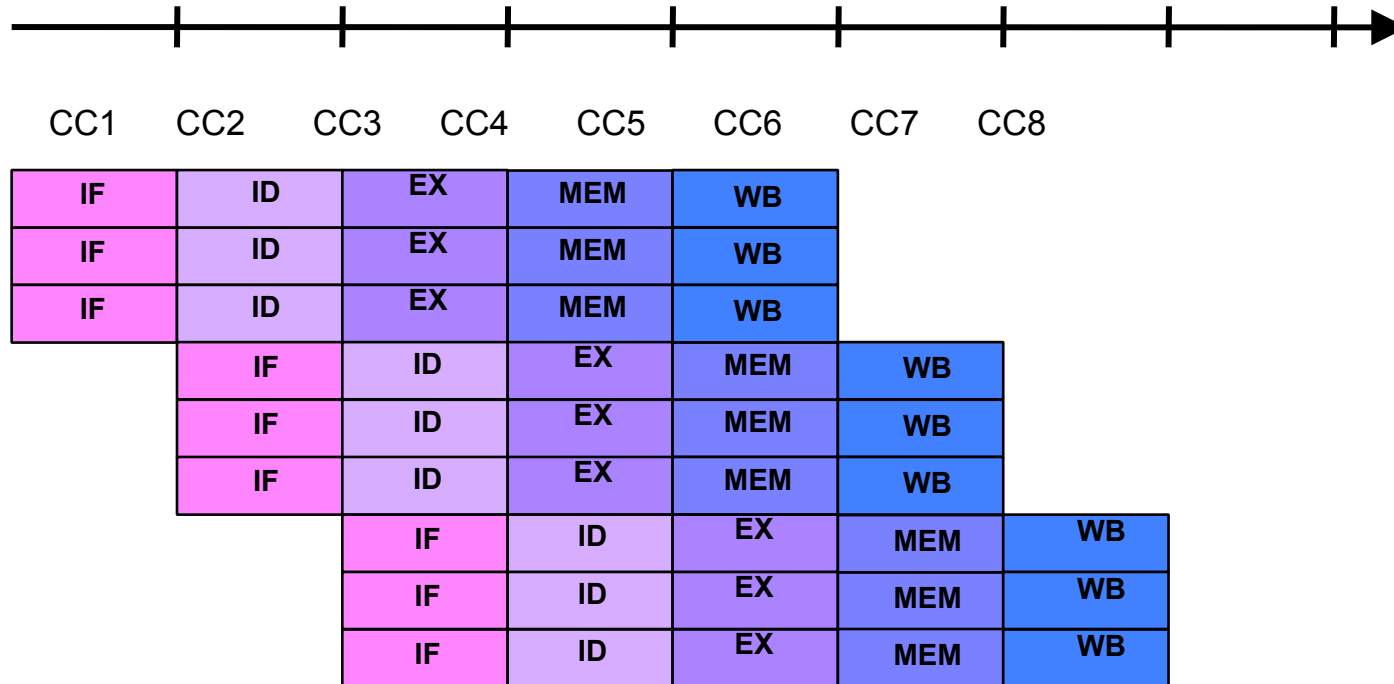


# Acelerando máquinas: *Superscalar*

- A estrutura *superscalar* contém múltiplas unidades funcionais que são capazes de fazer a mesma tarefa.
- Isto permite ao processador executar várias instruções similares concorrentemente, pelo roteamento das instruções às unidades de execução disponíveis.

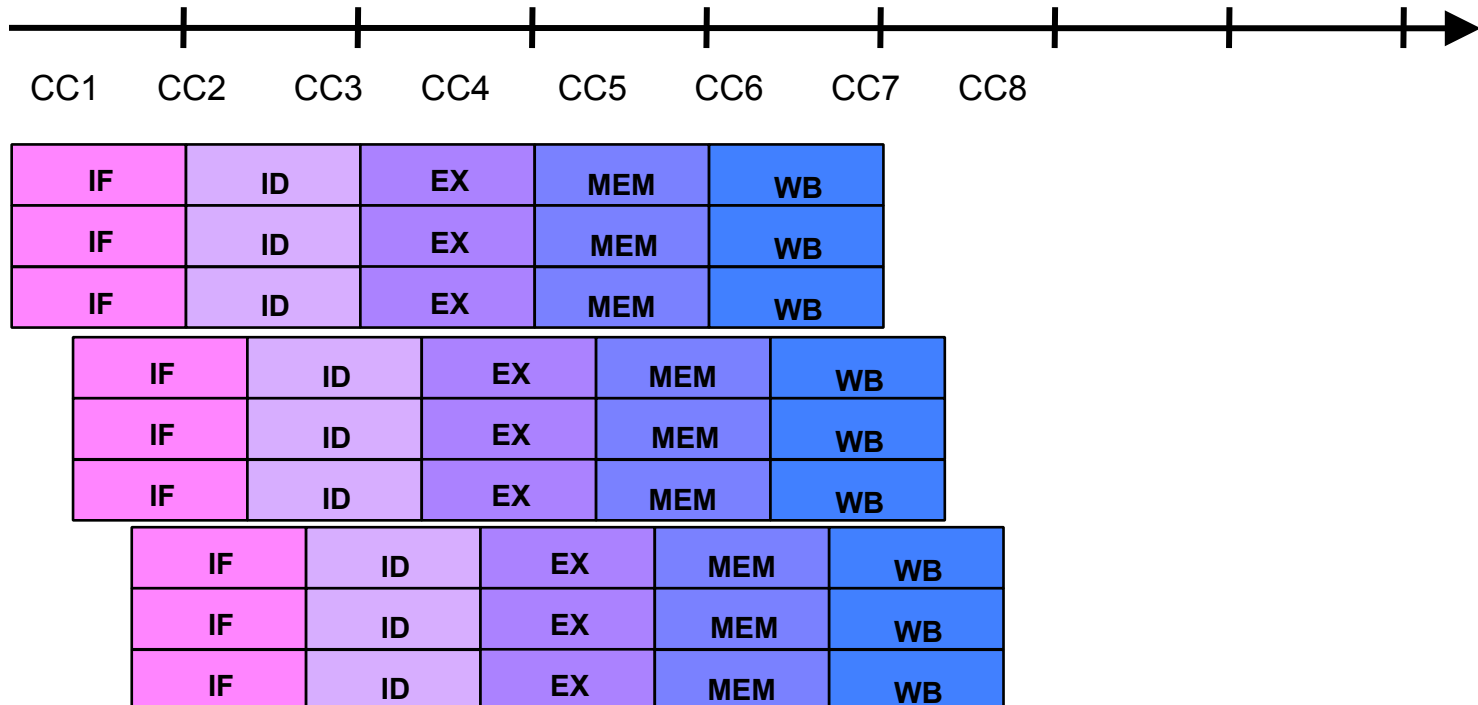


# Pipeline Superescalar





# Superpipeline Superescalar



Quantas instruções são iniciadas a cada ciclo de clock externo?



# Conclusão

- *Pipeline* aumenta o desempenho através do aumento da taxa de execução de instruções
  - Executa múltiplas instruções em paralelo
  - Cada instrução tem a mesma latência
- Conflitos afetam o desempenho do *pipeline*
  - Conflitos de dados, controle e estruturais
- Projeto da ISA afeta a complexidade da implementação